

2. What you do after work in your free time can depend on the weather. If it is sunny, you might choose to picnic with a friend, grab a drink with a colleague, or run errands. If it is raining, you might stay home and watch a movie instead. In this scenario, you have a clear outcome. In this case, that is classified as whether to "go out" or "stay in".

```
In [1]: # Importing Required Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics
import seaborn as sn
import matplotlib.pyplot as plt
```

```
In [2]: # Creating the Weather Activity Dataset
np.random.seed(42)
n_samples = 200

weather = np.random.choice(['Sunny', 'Rainy', 'Cloudy', 'Windy'], n_samples)
temperature = np.random.randint(15, 40, n_samples)
humidity = np.random.randint(20, 95, n_samples)
wind_speed = np.random.randint(0, 40, n_samples)

# Decision rule: Go out if sunny/cloudy with temp > 20, Low humidity, Low wind
label = []
for i in range(n_samples):
    if weather[i] in ['Sunny', 'Cloudy'] and temperature[i] > 22 and humidity[i] < 50:
        label.append(1) # Go Out
    elif weather[i] == 'Sunny' and temperature[i] > 25:
        label.append(1) # Go Out
    else:
        label.append(0) # Stay In

# Encode weather as numeric
weather_map = {'Sunny': 0, 'Rainy': 1, 'Cloudy': 2, 'Windy': 3}
weather_encoded = [weather_map[w] for w in weather]

data = pd.DataFrame({
    'weather': weather_encoded,
    'temperature': temperature,
    'humidity': humidity,
    'wind_speed': wind_speed,
    'label': label
})

print(data.shape)
data.head()
```

(200, 5)

```
Out[2]:
```

	weather	temperature	humidity	wind_speed	label
0	2	21	38	18	0
1	3	31	23	15	0
2	0	34	54	4	1
3	2	18	83	34	0
4	2	19	68	11	0

```
In [3]: # Checking for any null values in dataset
data.isnull().sum()
```

```
Out[3]: weather      0
temperature  0
humidity     0
wind_speed   0
label        0
dtype: int64
```

```
In [4]: # Assigning dependent and independent variables
feature_cols = ['weather', 'temperature', 'humidity', 'wind_speed']
x = data[feature_cols]
y = data.label
```

```
In [5]: # Splitting the dataset into Training and Testing Dataset
x_train, x_test, y_train, y_test = train_test_split(x, y,
                                                    test_size=0.2, random_state=5)

display(x_train.shape, y_train.shape, x_test.shape,
        y_test.shape)
```

```
(160, 4)
(160,)
(40, 4)
(40,)
```

```
In [6]: # Fitting the Model (Decision Tree Classifier)
model = DecisionTreeClassifier(criterion='entropy', random_state=5)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print('y_pred: ', y_pred)
```

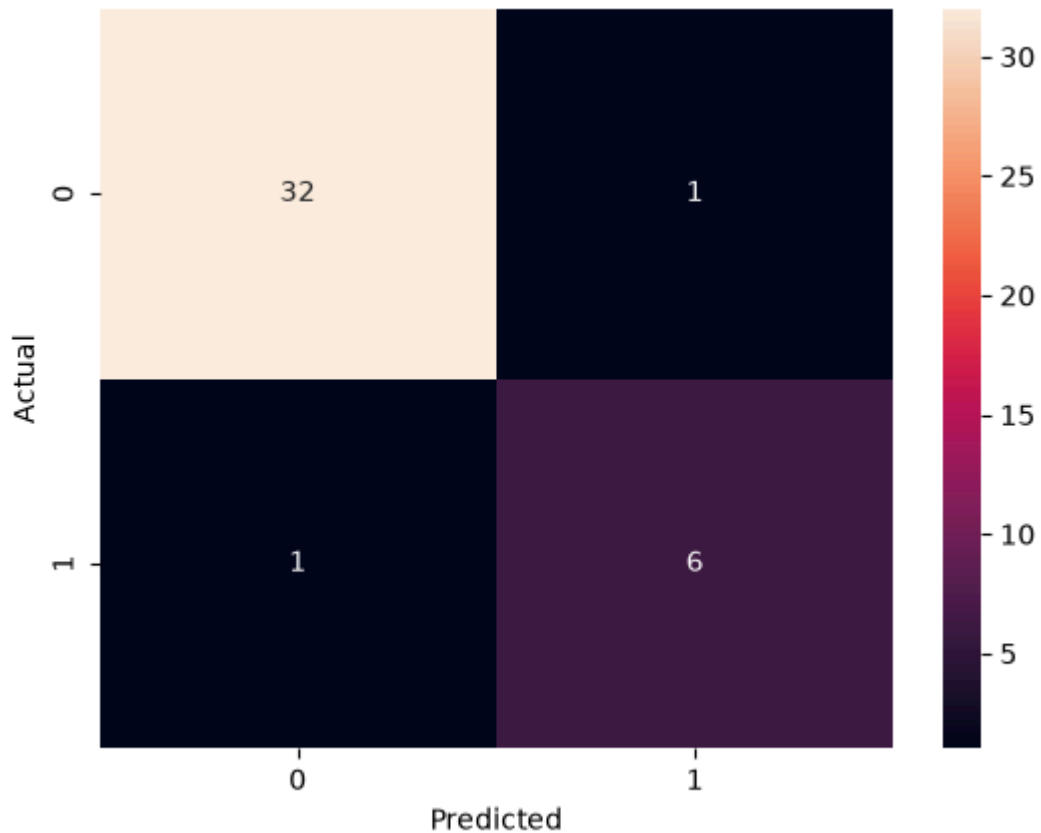
```
y_pred: [0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 1 0 0
0
1 0 0]
```

```
In [7]: # Evaluation Metrics
conf_mat = metrics.confusion_matrix(y_test, y_pred)
print('Confusion Matrix : ', conf_mat)
Accuracy_score = metrics.accuracy_score(y_test, y_pred)
print('Accuracy Score : ', Accuracy_score)
print('Accuracy in Percentage : ', int(Accuracy_score * 100), '%')
```

```
Confusion Matrix : [[32  1]
 [ 1  6]]
Accuracy Score : 0.95
Accuracy in Percentage : 95 %
```

```
In [8]: # Confusion Matrix Heatmap
conf_mat = pd.crosstab(y_test, y_pred, rownames=['Actual'],
                        colnames=['Predicted'])
sn.heatmap(conf_mat, annot=True)
```

Out[8]: <Axes: xlabel='Predicted', ylabel='Actual'>



```
In [9]: # Visualizing the Decision Tree
plt.figure(figsize=(20, 10))
tree.plot_tree(model, feature_names=feature_cols,
                class_names=['Stay In', 'Go Out'],
                filled=True, rounded=True)
plt.title('Decision Tree - Weather Activity Classifier')
plt.show()
```

